

MySQL



Introdução

- Os bancos de dados desempenham um papel fundamental na computação moderna, sendo essenciais para o armazenamento, organização e recuperação eficiente de dados em uma ampla variedade de contextos.
- Eles são a base sobre a qual sistemas de informação, aplicativos web, análises de dados e muitas outras soluções computacionais são construídas.
- A importância dos bancos de dados na computação reside no fato de que eles permitem que as organizações gerenciem grandes volumes de informações de forma estruturada, confiável e segura, fornecendo uma base sólida para tomada de decisões, automação de processos e inovação tecnológica.
- Além disso, os bancos de dados são essenciais para garantir a integridade, disponibilidade e segurança dos dados, aspectos críticos em um mundo onde a informação desempenha um papel central em praticamente todas as áreas da atividade humana.

Introdução ao MySQL

- O MySQL é um sistema de gerenciamento de banco de dados relacional (RDBMS) de código aberto amplamente utilizado em aplicativos web, sistemas de informação, e em ambientes empresariais.
- Ele desempenha um papel fundamental na construção de sistemas de informação, fornecendo uma estrutura robusta e escalável para armazenar, acessar e gerenciar dados de forma eficiente.

Funções do MySQL

Como parte de um sistema de informação, o MySQL desempenha várias funções importantes:

- 1) Armazenamento de dados:
- 2) Recuperação de dados:
- 3) Suporte a transações:
- 4) Segurança e controle de acesso
- 5) Escalabilidade e desempenho

Armazenamento de dados

- O MySQL fornece um local seguro e confiável para armazenar uma ampla variedade de dados, incluindo informações de clientes, transações, registros de atividades, inventários e muito mais.
- Ele permite que os dados sejam organizados em tabelas relacionadas, facilitando a recuperação e manipulação de informações de forma estruturada.

Recuperação de dados

- O MySQL permite que os usuários executem consultas complexas para recuperar dados específicos com base em critérios de pesquisa.
- Ele suporta uma ampla variedade de operações de consulta, incluindo filtragem, junção de tabelas, agregação e classificação, permitindo que os usuários obtenham as informações necessárias de forma eficiente.

Suporte a transações

- O MySQL oferece suporte a transações ACID, garantindo que as operações de banco de dados sejam executadas de forma confiável e consistentemente, mesmo em ambientes de alto volume e concorrência.
- Isso é crucial para garantir a integridade dos dados em sistemas de informação que lidam com transações complexas e simultâneas.

Segurança e controle de acesso

- O MySQL fornece recursos avançados de segurança, permitindo que os administradores controlem o acesso aos dados, protejam informações confidenciais e garantam a conformidade com regulamentos de privacidade e segurança.
- Ele suporta autenticação baseada em senha, criptografia de dados, controle de acesso granular e auditoria de atividades.

Escalabilidade e desempenho

- O MySQL oferece recursos para dimensionar operações à medida que as necessidades de armazenamento e processamento de dados crescem.
- Ele suporta replicação, particionamento e clustering para distribuir a carga e melhorar o desempenho em sistemas de informação com alto volume de dados e tráfego.

Características do MySQL



ESCALABILIDADE



CONFIABILIDADE



SUORTE A
TRANSAÇÕES



SEGURANÇA



FLEXIBILIDADE



CAPACIDADE DE
PERSONALIZAÇÃO

Estrutura do MySQL



- Descrever a estrutura interna do MySQL, incluindo bancos de dados, tabelas, índices, procedimentos armazenados, triggers, entre outros



- Explicar como os dados são organizados e armazenados no MySQL

Estrutura do MySQL

O MySQL é um sistema de gerenciamento de banco de dados relacional (RDBMS) que possui uma estrutura interna complexa composta por vários componentes:

- Servidor MySQL
- Armazenamento de Dados
- Otimizador de Consultas
- Cache de Consultas
- Conexões e Segurança
- Logs e Monitoramento

Servidor MySQL

- O servidor MySQL é o componente principal que gerencia todas as operações de banco de dados. Ele inclui um processo principal chamado "mysqld" que lida com a inicialização do servidor, aceita conexões de clientes, processa consultas, gerencia transações e executa outras tarefas relacionadas ao banco de dados.

Armazenamento de Dados

- O MySQL oferece suporte a vários mecanismos de armazenamento, incluindo o InnoDB, MyISAM, Memory, CSV, entre outros. Cada mecanismo de armazenamento tem sua própria estrutura interna para armazenar e recuperar dados de maneira eficiente, oferecendo diferentes recursos e otimizações para atender a diferentes requisitos de aplicativos.

Otimizador de Consultas

- O otimizador de consultas é responsável por analisar consultas SQL e determinar o plano de execução mais eficiente para recuperar os dados solicitados. Ele leva em consideração índices, estatísticas de tabela e outras informações para gerar um plano de execução que minimize o tempo de resposta da consulta.

Cache de Consultas

- O MySQL possui um cache de consultas interno que armazena consultas e resultados de consultas em memória, permitindo a recuperação mais rápida de dados frequentemente acessados. Isso ajuda a melhorar o desempenho em casos nos quais consultas idênticas são executadas repetidamente.

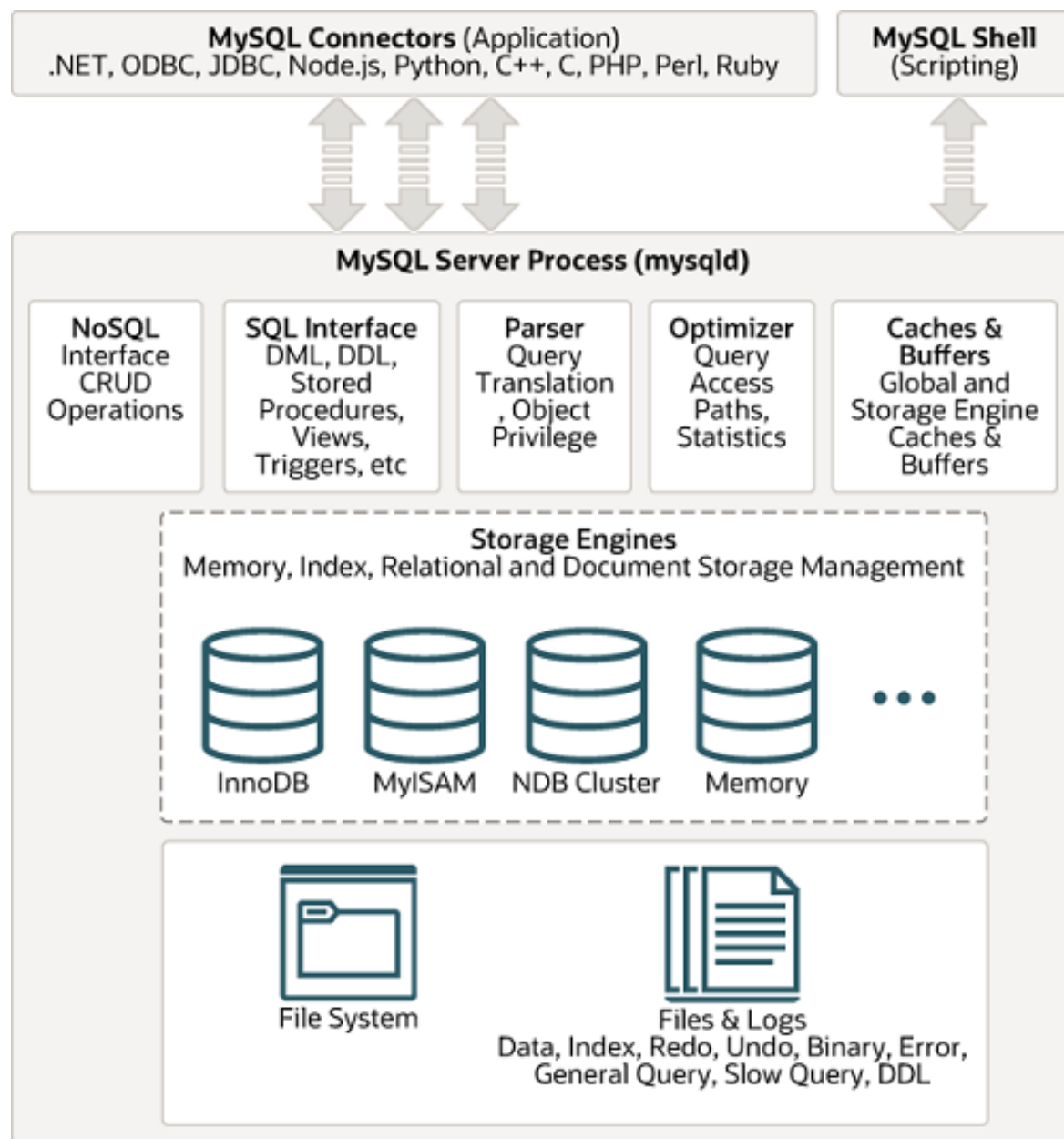
Conexões e Segurança

- O MySQL gerencia conexões de clientes e aplica políticas de segurança para controlar o acesso aos dados. Ele inclui um componente de autenticação e autorização que garante que apenas usuários autorizados possam acessar e modificar o banco de dados.

Logs e Monitoramento

- O MySQL gera vários tipos de logs, incluindo logs de consultas, logs de erros e logs de transações, que são usados para monitorar e diagnosticar o desempenho e a integridade do sistema. Além disso, o MySQL oferece suporte a várias ferramentas de monitoramento e administração para gerenciar o banco de dados de forma eficaz.

Arquitetura do MySQL



Linguagem SQL

Introdução à Linguagem
de Consulta Estruturada
(SQL)

Apresentar os principais
comandos do SQL
usados para manipular
dados, como SELECT,
INSERT, UPDATE, DELETE

Mostrar exemplos de
consultas SQL simples

Modelagem de Dados

A modelagem de dados desempenha um papel fundamental na construção de sistemas de informação, pois permite aos desenvolvedores e analistas representar e organizar os dados de forma estruturada e significativa. A importância da modelagem de dados pode ser compreendida através dos seguintes pontos:

Organização e estruturação dos dados

A modelagem de dados permite a organização dos dados em entidades, relacionamentos e atributos, proporcionando uma visão clara da estrutura dos dados e de como eles se relacionam entre si. Isso ajuda a estabelecer uma base sólida para a construção de sistemas de informação, garantindo a consistência e integridade dos dados.

Modelagem de Dados

Compreensão do negócio:

Através da modelagem de dados, é possível mapear os processos de negócio e as informações relevantes, o que ajuda a compreender as necessidades da organização e a identificar as relações entre diferentes entidades de dados. Isso contribui para a criação de sistemas de informação que atendam de forma eficaz às demandas do negócio.

Facilitação do desenvolvimento e manutenção de sistemas

Uma modelagem de dados bem definida serve como um guia para o desenvolvimento de sistemas de informação, fornecendo uma representação clara das estruturas de dados e seus relacionamentos. Isso reduz a ambiguidade e o risco de erros durante o desenvolvimento, além de facilitar a manutenção e evolução dos sistemas ao longo do tempo.

Modelagem de Dados

Melhoria da comunicação entre as partes interessadas

A modelagem de dados fornece uma linguagem comum para comunicação entre os desenvolvedores, usuários finais, analistas de negócios e outras partes interessadas. Isso ajuda a garantir que todos tenham uma compreensão compartilhada dos requisitos de dados e das necessidades do sistema de informação.

Integração de sistemas e interoperabilidade

Uma modelagem de dados consistente e bem projetada facilita a integração de sistemas e a troca de informações entre diferentes aplicativos e plataformas. Isso é particularmente importante em ambientes empresariais complexos, nos quais a interoperabilidade entre sistemas é essencial.

Modelagem de Dados

Ao considerar a importância da modelagem de dados na construção de sistemas de informação, é crucial enfatizar que uma modelagem bem executada pode contribuir significativamente para o sucesso e eficácia dos sistemas de informação, garantindo a qualidade, consistência e relevância dos dados utilizados pela organização.

Entidades

- Em modelagem de dados, uma entidade representa um objeto ou conceito do mundo real que pode ser diferenciado e identificado. Em um contexto de banco de dados, uma entidade geralmente corresponde a uma tabela, onde cada linha da tabela representa uma instância única dessa entidade. Por exemplo, em um sistema de gerenciamento de biblioteca, as entidades podem incluir "livro", "autor" e "usuário".

Atributos

- Os atributos são as características ou propriedades que descrevem uma entidade. Em um modelo de dados, os atributos são representados como colunas em uma tabela e descrevem as informações específicas associadas a cada instância da entidade. Por exemplo, os atributos de um livro podem incluir "título", "autor", "ano de publicação" e "ISBN".

Chave Primária

- Uma chave primária é um atributo (ou conjunto de atributos) que identifica exclusivamente cada registro em uma tabela. Ela serve como o principal identificador para as instâncias de uma entidade. A chave primária garante que não haja duplicação de registros e permite a identificação inequívoca de cada registro na tabela. Por exemplo, o número de identificação de um livro pode ser usado como a chave primária em uma tabela de livros.

Chave Estrangeira

- Uma chave estrangeira é um atributo em uma tabela que estabelece uma relação com a chave primária de outra tabela. Ela é usada para vincular as informações entre diferentes entidades em um banco de dados relacional. Por exemplo, em um sistema de biblioteca, a tabela de empréstimos pode ter uma chave estrangeira que se relaciona com a chave primária da tabela de usuários, indicando quem realizou o empréstimo.

Introdução ao MySQL Workbench

- **O que é MySQL Workbench?**
 - Ferramenta GUI desenvolvida pela Oracle para design, modelagem e administração de bancos de dados MySQL.
- **Principais Funcionalidades:**
 - Modelagem de dados.
 - Design visual de bancos de dados.
 - Migração de dados.
 - Execução de queries.
 - Administração do servidor.

Instalando o MySQL Workbench

- **Pré-requisitos:**

- **Windows/Mac/Linux:** Compatível com várias plataformas.
- **Download:** [MySQL Workbench Downloads](#).

- **Instalação:**

- **Windows:** Executar o instalador e seguir as instruções.
- **Linux:** Usar o gerenciador de pacotes (`apt-get`, `yum`) ou instalar manualmente.

Criando um Novo Modelo de Dados

- **Iniciando um Projeto**

- **Passos:**

1. Abra o MySQL Workbench.
2. Vá em ``File`` -> ``New Model``.
3. Uma nova janela de modelo será aberta.

- **Definindo Propriedades do Modelo**

- Nome do Modelo: Em ``Model`` -> ``Model Options``, defina o nome e outras propriedades.

Criando Tabelas

- **Adicionando uma Nova Tabela**

- **Passos:**

1. No painel esquerdo, selecione `EER Diagram`
2. Clique no ícone de tabela e arraste-o para o diagrama.
3. Dê um nome à tabela e defina suas colunas.

- **Definindo Colunas**

- **Nome da Coluna:** Defina um nome claro e descritivo.
- **Tipo de Dados:** Escolha entre tipos como `INT`, `VARCHAR`, `DATE`, etc.
- **Chave Primária:** Marque a coluna como chave primária se necessário.
- **Atributos de Coluna:** Defina atributos como `NOT NULL`, `AUTO_INCREMENT`

Definindo Relacionamentos

- **Tipos de Relacionamentos**

- **Um para Muitos (1:N):** Um registro em uma tabela pode estar relacionado com vários registros em outra tabela.
- **Muitos para Muitos (N:N):** Requer uma tabela associativa para armazenar as relações.

- **Criando Relacionamentos**

- **Passos:**

1. Selecione a ferramenta de relacionamento (linha com chave) no painel.
2. Clique na tabela de origem e depois na tabela de destino.
3. O MySQL Workbench cria a chave estrangeira automaticamente.

Relacionamentos

- **Edição de Relacionamentos**

- **Propriedades:**

- **Chave Estrangeira:** Editar o nome e colunas envolvidas.
 - **Restrição de Integridade:** Configurar **CASCADE** , **RESTRICT**, etc., para ações como DELETE e UPDATE.

Normalização de Dados

- **O que é Normalização?**

- Processo de organizar os dados para reduzir redundâncias e dependências.
- Melhora a integridade e eficiência do banco de dados.

- **Formas Normais:**

- **1ª Forma Normal (1NF):** Garantir que cada coluna contenha valores atômicos.
- **2ª Forma Normal (2NF):** Eliminar dependências parciais.
- **3ª Forma Normal (3NF):** Remover dependências transitivas.

- **Aplicação no MySQL Workbench**

- Estruture suas tabelas seguindo as regras de normalização.
- Utilize relacionamentos para garantir a integridade referencial.

Engenharia Reversa e Avançada

- **Engenharia Reversa**

- **Definição:** Importação de um banco de dados existente para criar um modelo visual.
- **Passos:**
 1. Vá em `Database-> Reverse Engineer`.
 2. Selecione a conexão com o banco de dados e siga os passos para importar o esquema.

- **Engenharia Avançada**

- **Definição:** Gerar o script SQL para criar um banco de dados a partir de um modelo.
- **Passos:**
 1. Em `Database`, selecione `Forward Engineer`.
 2. Siga as etapas para gerar e exportar o script SQL

Exportando e Compartilhando o Modelo

- **Exportação do Modelo**

- **Formatos Suportados:** PDF, PNG, SQL Script.

- **Passos:**

- 1. Vá em **File->Export** .

- 2. Selecione o formato desejado e configure as opções.

- **Colaboração**

- **Compartilhamento:** Exportar diagramas e modelos para colaborar com outros membros da equipe.

- **Integração com Versionamento:** Use sistemas como Git para versionar e compartilhar o modelo.

MySQLWorkbench - Considerações Finais

- **Próximos Passos:**

- Pratique criando modelos para diferentes tipos de sistemas.
- Explore funcionalidades avançadas como engenharia reversa e forward.

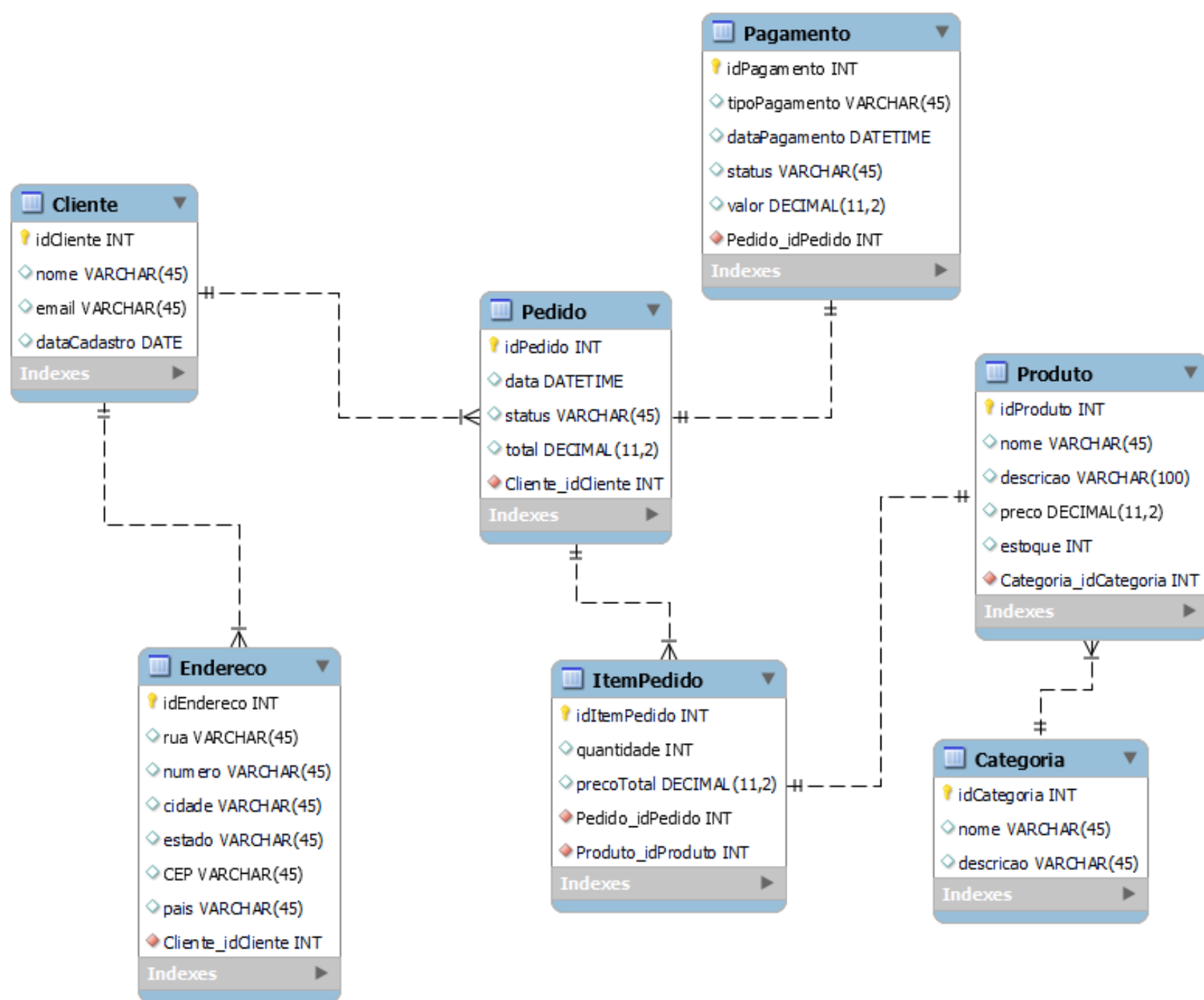
- **Recursos Adicionais:**

- Documentação Oficial do MySQL Workbench: [MySQL Workbench Documentation](#)

Exemplos Práticos

- **Caso de Uso 1:** Modelagem de um Sistema de E-commerce.
 - **Tabelas:** Produtos, Clientes, Pedidos, Pagamentos.
 - **Relacionamentos:** 1:N entre Clientes e Pedidos, N:N entre Produtos e Pedidos.
- **Caso de Uso 2:** Modelagem de um Sistema de Gestão Escolar.
 - **Tabelas:** Alunos, Professores, Turmas, Matrículas.
 - **Relacionamentos:** 1:N entre Turmas e Alunos, 1:N entre Professores e Turmas.

E-commerce



Integração com Linguagens de Programação

Apresentar a
integração do MySQL
com a linguagem de
programação Python

Mostrar como
conectar o MySQL a
uma aplicação web
ou desktop

Conectando uma Aplicação Python ao MySQL

Pré-Requisitos:

- **Python Instalado:** Versão 3.6 ou superior.
- **MySQL Server Instalado:** Pode ser local ou em um servidor remoto.
- **Biblioteca MySQL Connector:** Utilizaremos o `mysql-connector-python`

Instalando o MySQL Connector

- Comando de instalação:

```
pip install mysql-connector-python
```

- **Objetivo:** Essa biblioteca permite a conexão entre Python e MySQL, gerenciando a comunicação e a execução de comandos SQL.

Configurando o Banco de Dados MySQL

- Criando um banco de dados:

```
CREATE DATABASE exemplo_db;
```

- Criando uma tabela:

```
USE exemplo_db;
```

```
CREATE TABLE usuarios (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100),  
    email VARCHAR(100)  
);
```

Configurando o Banco de Dados MySQL

Inserindo dados iniciais:

```
INSERT INTO usuarios (nome, email) VALUES ('Alice', 'alice@example.com');  
INSERT INTO usuarios (nome, email) VALUES ('Bob', 'bob@example.com');
```

Conectando o Python ao MySQL

```
import mysql.connector

# Conectando ao banco de dados

conexao = mysql.connector.connect(
    host="localhost",
    user="seu_usuario",
    password="sua_senha",
    database="exemplo_db"
)

if conexao.is_connected():
    print("Conectado ao banco de dados com sucesso!")
```

Conectando o Python ao MySQL

•Explicação:

- `host`: Endereço do servidor MySQL (pode ser `localhost`).
- `user`: Nome de usuário do MySQL.
- `password`: Senha do usuário.
- `database`: Nome do banco de dados.

Executando Comandos SQL no Python

- Criando um cursor e executando uma consulta:

```
cursor = conexao.cursor()

cursor.execute("SELECT * FROM usuarios")

resultados = cursor.fetchall()

for linha in resultados:

    print(linha)
```

- `cursor.execute()`: Executa o comando SQL.
- `cursor.fetchall()`: Recupera todos os registros da consulta.

Inserindo Dados no Banco de Dados

```
sql = "INSERT INTO usuarios (nome, email) VALUES (%s, %s)"
valores = ("Carlos", "carlos@example.com")

cursor.execute(sql, valores)
conexao.commit()

print(f"{cursor.rowcount} registro inserido.")
```

- `conexao.commit()`: Confirma as alterações feitas no banco de dados.
- `cursor.rowcount`: Retorna o número de linhas afetadas.

Tratamento de Erros

```
try:
    # Código de conexão e execução
except mysql.connector.Error as err:
    print(f"Erro: {err}")
finally:
    if conexao.is_connected():
        cursor.close()
        conexao.close()
    print("Conexão encerrada.")
```

- O bloco `try-except` captura erros e garante que a conexão seja encerrada corretamente.

Encerrando a Conexão

```
cursor.close()  
conexao.close()
```

•**Importância:** Evita vazamentos de memória e garante a integridade do sistema.

Considerações Finais

- **Escalabilidade:** Uso de pooling de conexões para aplicações maiores.
- **Segurança:** Armazenamento seguro de credenciais (ex.: variáveis de ambiente).
- **Documentação:** Consultar a [documentação oficial](#) para funcionalidades avançadas.

Demonstração Prática

- **Passo-a-passo:**
 - Criação de um banco de dados.
 - Conexão e manipulação de dados usando Python.

Casos de Uso

- Exemplos de casos de uso do MySQL em um sistemas de e-commerce

Classe Database: construtor

```
import mysql.connector

from mysql.connector import Error


class Database:

    conexao = None

    def __init__(self):

        try:

            # Estabelecendo a conexão com o banco de dados

            self.conexao = mysql.connector.connect(

                host='localhost',          # Endereço do servidor MySQL

                database='ecommerce', # Nome do banco de dados

                user='root',              # Nome de usuário do MySQL

                password=''               # Senha do usuário do MySQL

            )

            if self.conexao.is_connected():

                db_info = self.conexao.get_server_info()

                print("Conectado ao servidor MySQL versão", db_info)

                cursor = self.conexao.cursor()

                cursor.execute("SELECT DATABASE();")

                nome_do_banco = cursor.fetchone()

                print("Conectado ao banco de dados", nome_do_banco)

        except Error as e:

            print("Erro ao conectar ao MySQL", e)
```

Classe Database: continuação

```
def fecha(self):  
    if self.conexao.is_connected():  
        self.conexao.close()  
        print("Conexão ao MySQL encerrada")  
  
def insereCategoria(self, categoria):  
    if self.conexao.is_connected():  
        cursor = self.conexao.cursor()  
        # Inserir um novo registro  
        sql_insert = "INSERT INTO Categoria (nome, descricao) VALUES (%s, %s)"  
        valores = (categoria.nome, categoria.descricao)  
        cursor.execute(sql_insert, valores)  
        # Confirmar a transação  
        self.conexao.commit()  
        print("Registro inserido com sucesso")  
  
def listaCategorias(self):  
    if self.conexao.is_connected():  
        cursor = self.conexao.cursor()  
        cursor.execute("SELECT * FROM Categoria;")  
        registros = cursor.fetchall()  
        print("Total de registros encontrados:", cursor.rowcount)  
        for linha in registros:  
            print("id:", linha[0], "nome:", linha[1], 'descrição:', linha[2])  
        return registros
```

Classe Categoria

```
class Categoria:

    def __init__(self: object, nome: str, descricao: str) -> None:

        self.__id: int = id

        self.__nome: str = nome

        self.__descricao: str = descricao

    @property
    def id(self):

        return self.__id

    @property
    def nome(self):

        return self.__nome

    @property
    def descricao(self):

        return self.__descricao
```

Executando Database

```
db = Database()
categoria = Categoria("CatName", "CatDesc")
db.insereCategoria(categoria)
lista = db.listaCategorias()
db.fecha()
```

Introdução ao Flask em Python

O que é Flask?

- **Flask** é um microframework web em Python.
- Desenvolvido por **Armin Ronacher**.
- Enfatiza a simplicidade, flexibilidade e controle sobre o processo de desenvolvimento.
- "**Micro**" significa que Flask visa manter o núcleo simples, mas extensível.

Características Principais do Flask

- **Leve e Minimalista:** Focado em fornecer apenas as funcionalidades essenciais.
- **Extensível:** Fácil de adicionar extensões para funcionalidades extras, como banco de dados ou autenticação.
- **Servidor de Desenvolvimento:** Inclui um servidor integrado para testes rápidos.
- **Suporte a URLs Dinâmicos:** Facilita a criação de URLs limpas e amigáveis.
- **Compatível com Jinja2:** Utiliza o motor de templates Jinja2 para renderizar HTML.

Instalando o Flask

1. Verifique se o Python está instalado:

```
python --version
```

2. Instale o Flask via pip:

```
pip install Flask
```

3. Verifique a instalação:

```
python -m flask --version
```

Seu Primeiro Projeto Flask

1. Crie um arquivo Python, por exemplo, `app.py`:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello, World!'
```

2. Execute o aplicativo:

```
flask run
```

3. Acesse no navegador: <http://127.0.0.1:5000>

Roteamento no Flask

- **Roteamento** define como as URLs do seu aplicativo mapeiam para funções específicas.
- Exemplo de rotas dinâmicas:

```
@app.route('/user/<username>')
def show_user_profile(username):
    return f'Profile: {username}'

@app.route('/post/<int:post_id>')
def show_post(post_id):
    return f'Post {post_id}'
```

Templates com Jinja2

- Jinja2 permite criar HTML dinâmico.
- Exemplo de template (`templates/index.html`):

```
<!doctype html>
<html>
  <head>
    <title>{{ title }}</title>
  </head>
  <body>
    <h1>{{ heading }}</h1>
    <p>{{ content }}</p>
  </body>
</html>
```

- Renderizando o template no Flask:

```
from flask import render_template

@app.route('/')
def home():
    return render_template('index.html', title="Home", heading="Welcome", content="This is Flask!")
```

- Recebendo os dados no Flask:

```
from flask import request

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        return f'Welcome, {username}!'
    return render_template('login.html')
```

Formulários e Requisições

- Flask facilita a manipulação de dados de formulários.
- Exemplo de formulário HTML:

```
<form method="POST">  
  <input type="text" name="username">  
  <input type="submit" value="Submit">  
</form>
```

Gerenciamento de Configurações

- Flask permite diferentes configurações para diferentes ambientes (desenvolvimento, produção, etc.).
- Exemplo de configuração:

```
app.config['DEBUG'] = True  
app.config['SECRET_KEY'] = 'supersecretkey'
```

- Configurações baseadas em ambiente:

```
app.config.from_object('config.DevelopmentConfig')
```

Recursos Adicionais

- **Documentação Oficial:** [Flask Documentation](#)
- **Tutoriais:** Flask Mega-Tutorial, Flask by Example.
- **Comunidade:** GitHub, Stack Overflow.

Extensões Flask

- **Flask-WTF**: Suporte avançado para formulários.
- **Flask-Login**: Gerenciamento de autenticação de usuários.
- **Flask-Migrate**: Migrações de banco de dados.
- **Flask-Mail**: Envio de emails.
- Instalação de extensões:

```
pip install Flask-WTF Flask-Login Flask-Migrate Flask-Mail
```

Deploy de Aplicações Flask

- **Servidor de Produção:** Use servidores como Gunicorn ou uWSGI.
- **Exemplo de uso com Gunicorn:**

```
pip install gunicorn  
gunicorn -w 4 app:app
```

- **Deploy na Nuvem:** Plataformas como Heroku, AWS, ou Google Cloud podem ser usadas para deploy.
- **Docker:** Containerize seu aplicativo Flask para facilitar o deploy.

Conclusão

- Flask é uma ferramenta poderosa e flexível para desenvolvimento web em Python.
- Ótimo para aplicações de pequeno a médio porte, mas pode escalar com a adição de extensões.
- Incentive a prática com pequenos projetos para dominar o framework.

Índices, Stored Procedures e Triggers

- Apresentar os recursos do MySQL

Índices, Stored Procedures e Triggers no MySQL

- **Índices:** Estruturas de dados que melhoram a velocidade das operações de busca no banco de dados.
- **Stored Procedures:** Conjunto de comandos SQL armazenados que podem ser executados no servidor.
- **Triggers:** Ações automáticas que são executadas em resposta a eventos específicos em uma tabela.

Índices no MySQL

- **O que são Índices?**

- Estruturas que armazenam uma pequena parte dos dados de uma tabela em uma forma de fácil acesso, geralmente usada para acelerar consultas.

- **Tipos de Índices**

- **Primary Key:** Índice único, automaticamente criado quando uma chave primária é definida.
- **Unique Index:** Garante que todos os valores em uma coluna sejam distintos.
- **Fulltext Index:** Usado para buscas textuais complexas.
- **Composite Index:** Índice que abrange várias colunas.

Índices no MySQL

- **Criação de Índices**

- **Exemplo:** Criar um índice simples na coluna `nome` da tabela `clientes`.

```
CREATE INDEX idx_nome ON clientes(nome);
```

- **Vantagens e Desvantagens**

- **Vantagens:**

- Acelera consultas, especialmente em grandes volumes de dados.
- Facilita a ordenação e filtragem.

- **Desvantagens:**

- Consome espaço adicional no disco.
- Pode desacelerar operações de inserção, atualização e exclusão.

Stored Procedures no MySQL

- **O que são Stored Procedures?**
 - Conjuntos de comandos SQL pré-compilados armazenados no banco de dados, que podem ser chamados repetidamente com parâmetros específicos.
- **Vantagens das Stored Procedures**
 - **Performance:** Evita a recompilação frequente de comandos SQL
 - **Reutilização:** Código modular e reutilizável.
 - **Segurança:** Restringe o acesso direto às tabelas.

Stored Procedures

- **Criação de uma Stored Procedure**
 - **Exemplo:** Criar uma procedure para inserir um novo cliente:

```
DELIMITER $$  
  
CREATE PROCEDURE add_cliente(IN nome_cliente VARCHAR(100), IN idade_cliente INT)  
  
BEGIN  
  
    INSERT INTO clientes (nome, idade) VALUES (nome_cliente, idade_cliente);  
  
END $$  
  
DELIMITER ;
```

Stored Procedures

- Chamando uma Stored Procedure
 - Exemplo:

```
CALL add_cliente('João Silva', 30);
```

Stored Procedures

- **Stored Procedures com Saída**

- **Exemplo:** Criar uma procedure que retorna a idade de um cliente:

```
DELIMITER $$  
  
CREATE PROCEDURE get_idade_cliente(IN nome_cliente VARCHAR(100), OUT idade_cliente  
INT)  
  
BEGIN  
  
    SELECT idade INTO idade_cliente FROM clientes WHERE nome = nome_cliente;  
  
END $$  
  
DELIMITER ;
```

```
CALL get_idade_cliente('João Silva', @idade);  
  
SELECT @idade;
```

Triggers no MySQL

- **O que são Triggers?**

- Blocos de código que são automaticamente executados em resposta a eventos como `INSERT`, `UPDATE` ou `DELETE` em uma tabela.

- **Tipos de Triggers**

- **Before Triggers:** Executados antes da operação.
- **After Triggers:** Executados após a operação.

- **Aplicações Comuns de Triggers**

- **Auditoria:** Registrar alterações ou acessos aos dados.
- **Validação:** Impedir operações ilegais ou inconsistentes.
- **Automação:** Atualizar valores derivados ou dependentes.

Triggers

- **Criação de um Trigger**

- **Exemplo:** Criar um trigger que registra alterações em uma tabela de logs:

```
CREATE TRIGGER before_cliente_update
BEFORE UPDATE ON clientes
FOR EACH ROW
BEGIN
    INSERT INTO cliente_logs (cliente_id, data_alteracao, detalhes)
    VALUES (OLD.id, NOW(), CONCAT('Mudança no nome de ', OLD.nome, ' para ', NEW.nome));
END;
```

Boas Práticas e Considerações

- **Índices**

- Use índices em colunas frequentemente usadas em cláusulas `WHERE`, `JOIN` e `ORDER BY`.
- Evite criar muitos índices em tabelas com muitas operações de escrita.

- **Stored Procedures**

- Escreva procedures com lógica clara e modular.
- Evite colocar muita lógica de negócios no banco de dados para não dificultar a manutenção.

- **Triggers**

- Use triggers com moderação, pois podem complicar a lógica e dificultar a depuração.
- Certifique-se de que os triggers não introduzam loops ou problemas de desempenho.

Exercícios Práticos

1. **Índices:** Comparar tempos de execução de consultas com e sem índice.
2. **Stored Procedures:** Criar uma procedure que gera relatórios mensais.
3. **Triggers:** Implementar um trigger para auditoria automática de registros.

Administração do MySQL

Discutir tarefas de administração do MySQL, como criação e manutenção de bancos de dados, gerenciamento de usuários, backup e restauração de dados

Destacar a importância da segurança e da manutenção regular do banco de dados

Gerenciamento de Usuários

- Criação de Usuários

```
CREATE USER 'novo_usuario'@'localhost' IDENTIFIED BY 'senha';
```

- Concessão de Privilégios

```
GRANT ALL PRIVILEGES ON banco_de_dados.* TO 'novo_usuario'@'localhost';
```

Gerenciamento de Usuários

- **Exibição de Privilégios**

```
SHOW GRANTS FOR 'usuario'@'localhost';
```

- **Revogação de Privilégios**

```
REVOKE INSERT ON banco_de_dados.* FROM 'usuario'@'localhost';
```

Gerenciamento de Bancos de Dados

- Criação de um Banco de Dados

```
CREATE DATABASE nome_do_banco;
```

- Listagem de Bancos de Dados

```
SHOW DATABASES;
```

Gerenciamento de Bancos de Dados

- Backups

```
mysqldump -u root -p nome_do_banco > backup.sql
```

- Restauração de Banco de Dados

```
mysql -u root -p nome_do_banco < backup.sql
```

Manutenção e Otimização

- **Monitoramento de Performance**

```
SHOW STATUS;
```

- Ferramentas: MySQL Workbench, Percona Monitoring and Management (PMM).

- **Otimização de Consultas**

- Uso de índices: `CREATE INDEX idx_coluna ON tabela (coluna);`

- Análise de consultas: `EXPLAIN SELECT * FROM tabela WHERE coluna = 'valor';`

Manutenção e Otimização

- Tabelas e Índices
 - Análise e Otimização:

```
ANALYZE TABLE nome_tabela;  
OPTIMIZE TABLE nome_tabela;
```

Segurança

- **Contas e Privilégios**
 - **Privilégios mínimos:** Apenas o necessário para cada usuário.
- **Proteção Contra Injeção SQL**
 - **Boas práticas:**
 - Uso de queries parametrizadas.
 - Validação rigorosa de dados de entrada.
- **Criptografia**
 - **SSL/TLS:** Configuração de comunicação segura entre cliente e servidor.

Backup e Recuperação

- **Tipos de Backup**

- **Completo:** Backup de todo o banco de dados.
- **Incremental:** Apenas as alterações desde o último backup.

- **Automatização de Backups**

- Agendamento com **cron** (Linux)

```
0 2 * * * /usr/bin/mysqldump -u root -p nome_do_banco > /caminho_para_backup/backup.sql
```

- **Recuperação de Dados**

- **Recuperação a partir de backups:**

```
mysql -u root -p nome_do_banco < caminho_para_backup/backup.sql
```

Monitoramento e Tuning

- **Monitoramento Contínuo**

- **Ferramentas:** MySQL Workbench, PMM, Nagios.
- **Métricas Importantes:**
 - **Uso de CPU e Memória.**
 - **Conexões Ativas**
 - **Desempenho de Consultas.**

- **Tuning de Performance**

- **Ajuste de Parâmetros:**
 - `innodb_buffer_pool_size`: Ajuste para melhor uso de memória.
 - `query_cache_size`: Configuração de cache de consultas (se necessário).

Atualização e Migração

- **Atualização do MySQL**

- **Backup Completo** antes de atualizar
- **Verificação de Compatibilidade:**
 - Comando: `mysql_upgrade`

- **Migração**

- **Migrar entre Servidores:**
 - Ferramentas: `mysqldump`, `mysqlpump` ou replicação.

- **Teste Pós-Migração**

- Verificar integridade dos dados.
- Testar desempenho e ajuste.

Ferramentas Auxiliares

- **MySQL Workbench**
 - Design de banco de dados, administração, e visualização.
- **phpMyAdmin**
 - Interface web para administração do MySQL.
- **Percona Toolkit**
 - Conjunto de ferramentas para auditoria, diagnóstico e otimização.

Anotações Finais

- **Recursos Adicionais:**

- Documentação Oficial do MySQL: [MySQL Documentation](#)
- Comunidades e Fóruns: Stack Overflow, Percona, etc.

Recursos e Comunidade

- Destacar os recursos disponíveis para aprender mais sobre o MySQL, como documentação oficial, fóruns de suporte, tutoriais online, entre outros
- Encorajar os alunos a explorar a comunidade MySQL e a contribuir para o desenvolvimento do software